

Relatório Do Projeto 2 de P.O.O

Jogo de Xadrez

1st Lucas Araújo Campos Szuster

DELT
UFMG

Belo Horizonte, Brasil
lucasacszuster@gmail.com

2nd Arthur Coronho Seabra Eiras

DELT
UFMG

Belo Horizonte, Brasil
arthurcse@ufmg.br

3rd Caio César Araújo Guimarães

DELT
UFMG

Belo Horizonte, Brasil
caio.araujog@gmail.com

Abstract—Esse documento explica e especifica a implementação do clássico jogo de xadrez para terminal na linguagem de Python. Ao longo do texto, a arquitetura do projeto será especificada, assim como suas classes com seus métodos associados e as estratégias que usamos para ter nosso produto final.

Index Terms—peça, tabuleiro, peão, rainha, bispo, rei, cavalo, torre.

I. INTRODUÇÃO

Para o segundo projeto de POO, escolhemos implementar o clássico jogo de xadrez na linguagem **Python**. A ideia foi construir um sistema capaz de simular no terminal um jogo completo de xadrez, com movimentos equivalentes aos de xadrez real. Uma das grandes inspirações para o projeto é o site **chess.com** onde é possível jogar xadrez com pessoas do mundo todo. Dado o escopo do nosso projeto, nos focamos na realização do jogo corretamente como ponto principal. Dessa forma, procuramos ao máximo aplicar os conceitos de POO bem como os aprendizados em sala de aula para implementar o jogo de xadrez.

II. ARQUITETURA DO PROJETO

A arquitetura do projeto se baseou em alguns pilares:

A. Princípios da POO

Alguns dos princípios fundamentais da **POO** são dados pela sigla **SOLID** que foi usada extensivamente em nosso projeto:

- **S**: Mantemos o princípio da responsabilidade única com classes distintas com escopos bem definidos.
- **O**: O princípio do aberto/fechado foi usado nos atributos e métodos das classes colocando os atributos como privados e os métodos públicos.
- **L**: É possível ver o princípio de substituição de Liskov na classe Peça onde suas classes filhas podem substituí-la em determinada situação sem alterar o funcionamento do código.
- **I**: O princípio de segregação de interfaces também foi utilizado já que todas as classes implementam métodos usados somente por elas, ou seja, não há métodos excedentes ou inativos.

- **D**: Por fim, o princípio de inversão de dependência foi aplicado para abstrair e simplificar o problema, sendo um claro exemplo a classe peça que implementa uma abstração de alto nível e define o comportamento e escopo para suas sub-classes

B. Diagrama UML

O diagrama UML^[1] foi fundamental para a visualização e elaboração do projeto de uma maneira mais prática tornando as classes e suas relações mais claras. Apesar do diagrama ter sido modificado substancialmente, o objetivo inicial do projeto se manteve e o uso do **UML** tornou mais fácil a implementação e desenvolvimento do sistema.

C. Métodos de Engenharia de Sistemas

Nesse projeto, utilizamos um pouco dos conhecimentos adquiridos na disciplina de Processos e Métodos onde temos os métodos de Engenharia de Sistemas que são uma base para o desenvolvimento e estruturação do ciclo de vida de um sistema^[2]. Esses métodos são divididos em 4 fases sequenciais sendo elas:

1) *Análise de Requisitos*: A análise de requisitos é baseada em compreender as entradas ainda que incompletas e corrigir quando necessário. A parte chave dessa fase é a organização e interpretação das entradas de forma a compreendê-las e dar uma direção confiável ao sistema. Simplificando em uma frase, análise de requisitos é sobre clarificar, corrigir e quantificar. No nosso projeto, os requisitos demandados foram:

- O sistema precisa simular com precisão as peças de xadrez e seus movimentos correspondentes.
- O tabuleiro deve ser composto por uma matriz 8x8 com colunas numeradas de 1 a 8 e linhas numeradas da mesma forma.
- O escopo do programa deve abranger somente vitórias por Xequemate, sendo excluídas vitórias por tempo
- O sistema deve ser capaz de distinguir adequadamente as peças de cada jogador.

2) *Definição Funcional*: A definição funcional é a fase de organização das funções do sistema, isto é quais são os blocos funcionais do sistema e como eles se relacionam entre si e em quais arranjos eles podem ser colocados de forma a beneficiar

o sistema como um todo. Em nossa implementação usamos os seguintes blocos funcionais:

- Peças
- Jogadas
- Progressão no tabuleiro
- Final de jogo

3) *Definição Física*: A definição física é a etapa responsável pela definição da produção do *software* em si sendo possível a aplicação prática das partes do sistema, ou seja das classes e métodos do sistema de fato. Para o nosso projeto definimos que a interface seria o próprio terminal que disponibilizaria na tela o tabuleiro a cada jogada e, a partir dele utilizamos as demais classes para realizar cada jogada e corretamente e disponibilizar o resultado ao final caso os critérios de xoque-mate sejam atendidos.

4) *Validação de Projeto*: A validação de projeto é a etapa final dos métodos e possui como objetivo a validação dos requisitos iniciais e das métricas de desempenho do sistema. Uma parte crucial dessa fase é a análise de dados onde os componentes do sistema são testados para detectar erros ou pontos fracos assim contribuindo para a integridade e cumprimento dos requisitos. No nosso projeto, fizemos vários jogos e jogadas simuladas para testar a precisão do sistema ao realizar vários jogos de xadrez, alterando as jogadas e peças utilizadas foi possível verificar a acurácia além de corrigir seus erros.

III. CLASSES DO PROJETO

As principais classes responsáveis pelo funcionamento do projeto são:

A. Peça

A classe peça é uma estrutura de dados abstrata que se ramifica em todos os tipos de peças presentes no xadrez e consequentemente implementadas no projeto. Elas são:

- Peão
- Bispo
- Cavalo
- Torre
- Rainha
- Rei

Cada uma dessas peças possui suas peculiaridades, formas de movimentação e de tomada de peças. Em razão disso, foram implementadas classes, herdadas de peça para cada uma delas. Cada peça implementada possui então um classe de nome análogo. A superclasse **Peça**, que resulta nas classes filhas, possui atributos, fazendo com que todas as peças compartilhem as seguintes características: ser responsável por conter seu ícone, sua cor, sua coordenada atual, uma instância de tabuleiro e uma lista de possíveis movimentos. **Peça** possui todos os getters e setters análogos visando as boas práticas de encapsulamento da Programação Orientada à Objetos. Há alguns métodos que todos os tipos de peça compartilham. Eles são:

Tentar Adicionar: Recebe um inteiro para linha e outro para coluna. A função checa se a coordenada recebida é

diferente da coordenada atual da peça, se ela já não está na lista de possíveis movimentos da peça e, caso haja uma peça se o tipo daquela presente lá é diferente da peça que está movimentando, visto que no xadrez uma peça de uma cor não pode tomar uma peça da mesma cor. Caso todas essas checagens forem válidas, ela adiciona a coordenada na lista de possíveis movimentos.

Pode Se Mover Para Ca: Esse método é mantido e utilizado dessa forma em todas as classes, exceto em peão, que realiza um override no método por motivos a serem detalhados mais a frente. Para casos gerais, ela recebe uma coordenada e caso ela estiver na lista de possíveis movimentos, retorna verdadeiro, indicando que a peça pode se mover para cá. Caso contrário, retorna falso, ou seja, a coordenada recebida não está na lista de possíveis movimentos e a peça não poderá se mover para tal posição.

Seguem agora as especificações das classes que herdam de peça, com suas funcionalidades, atributos e principais métodos:

B. Peão

Assim como qualquer outra peça, um peão tem como atributos seu ícone, sua coordenada, uma lista de possíveis movimentos, uma cor e um tabuleiro. Em um jogo de Xadrez, um peão pode andar duas casas verticais em seu primeiro movimento e 1 casa vertical durante todo o resto do jogo. No entanto, ele não pode tomar peças que estão na sua frente. Ele sempre come na diagonal, diferente das outras peças que podem tomar peças que estão na sua lista de possíveis movimentos. Se há uma peça, mesmo que de outra cor, na frente do peão, ele ficará imóvel e não poderá a tomar. Caso ela esteja na sua diagonal esquerda ou direita, mesmo não podendo se movimentar naturalmente na ausência de uma peça para lá, o peão poderá tomar a peça, visto que sua tomada é sempre dada pelas diagonais ascendentes. Por isso, além de conter os atributos já ditos, o peão também possui uma variável do tipo booleano chamada *Ja_Movimentou* e uma lista para tomadas, visto que é uma lista diferente da de movimentos, já que o peão só poderá ir para as posições diagonais no caso de uma peça oposta^[3]. Em razão disso, os métodos para **Peão** são:

Tentar Adicionar: Esse método é análogo ao implementado na classe mãe, entretanto, tem que ser sobrecarregado pois como o peão não pode tomar em casas frontais, a última condição é levemente diferente. Ou seja, ele checa se há ou não uma peça na sua frente, não sua cor ser diferente da do peão, já que geralmente, se uma peça vai para uma posição do tabuleiro e há uma peça adversária ela é tomada. Como já explicado anteriormente, no caso do peão isso não pode acontecer.

Atualizar lista de possíveis coordenadas: Esse método checa qual a cor da peça e, a depender do retorno chama **Atualizar lista de possíveis coordenadas do peão branco** ou **Atualizar lista de possíveis coordenadas do peão preto**, em todos os casos passando a linha atual e a coordenada atual do peão.

Atualizar lista de possíveis coordenadas do peão branco:

Como o tabuleiro é uma matriz, cada lado do tabuleiro tem que se mover de maneira oposta. Por isso, as peças brancas se movem de maneira decremental, e para o caso do peão é importante, já que só anda para frente. Para isso, a função checa se o peão já se movimentou na partida, para averiguar se ele pode andar duas casas ou não. Após essa checagem, ele verifica se a linha na qual o peão está indo é igual a linha atual - 1 (ou - 2 a depender do já movimentou) e a coluna se mantém.

Atualizar lista de possíveis coordenadas do peão preto:

Como explicado anteriormente e pelo fato do tabuleiro ser uma matriz, cada lado do tabuleiro tem que se mover de maneira oposta. Por isso, as peças pretas se movem de maneira incremental, e para o caso do peão é importante, já que só anda para frente. Para isso, a função checa se o peão já se movimentou na partida, para averiguar se ele pode andar duas casas ou não. Após essa checagem, ele verifica se a linha na qual o peão está indo é igual a linha atual + 1 (ou + 2 a depender do já movimentou) e a coluna se mantém.

Tentar Adicionar Tomada: Método que realiza função semelhante ao Tentar Adicionar. Entretanto, o peão necessita de uma verificação a mais, em razão de sua tomada diferenciada. Essa função recebe uma linha e uma coluna e Adiciona coordenada à lista de movimentos para tomada somente se for válida no tabuleiro, específico para tomadas do peão. Para isso ele checa se a coordenada já não está nos movimentos para tomadas, se é diferente da coordenada atual e se a cor da peça na casa é diferente da atual.

Atualizar lista de coordenadas para tomada: Somente apresenta para o peão, essa função checa a cor da peça atual e chama **Atualizar listas de coordenada para tomada branca** ou **Atualizar listas de coordenada para tomada preta**. Como no peão além da checagem ordinária da movimentação também são feitas checagens nas diagonais adjacentes, a classe possui métodos análogos aos já descritos anteriormente mas que checam tais diagonais. Por isso:

Atualizar listas de coordenada para tomada branca:

Como o tabuleiro é uma matriz, cada lado do tabuleiro tem que se mover de maneira oposta. Por isso, as peças brancas se movem de maneira decremental, e para o caso do peão é importante, já que suas tomadas são realizadas nas diagonais adjacentes. Para isso, ele verifica se a linha na qual o peão está indo é igual a linha atual - 1 e a coluna + 1 ou - 1. Ou seja, garante que no caso da lista de movimentos para tomada, o peão não irá movimentar para frente nem para as diagonais de trás.

Atualizar listas de coordenada para tomada preta: Como o tabuleiro é uma matriz, cada lado do tabuleiro tem que se mover de maneira oposta. Por isso, as peças pretas se movem de maneira incremental, e para o caso do peão é importante, já que suas tomadas são realizadas nas diagonais adjacentes. Para isso, ele verifica se a linha na qual o peão está indo é igual a linha atual + 1 e a coluna + 1 ou - 1. Ou seja, garante que no caso da lista de movimentos para tomada, o peão não irá movimentar para frente nem para as diagonais de trás.

Pode se mover para cá: A classe **Peão** sobrescreve o método contido em classe justamente pela diferença na tomada e na movimentação natural. Por isso, a função recebe uma coordenada de destino, atualiza as listas de coordenadas do peão e checa se a coordenada destino está contida na lista de possíveis movimentos. Caso esteja mas haja uma peça, a função retorna falso, caso esteja e não haja uma peça, ela retorna verdadeiro. No caso da coordenada de destino não estar presente na lista de possíveis movimentos, há outra checagem se ela está na lista de movimentos para tomadas. Caso esteja, checa se há uma peça lá e se sua cor é diferente da cor da peça. Caso seja, retorna verdadeiro evidenciado que o peão pode ir para tal lugar. Caso as checagens não forem verdadeiras, o método retorna falso.

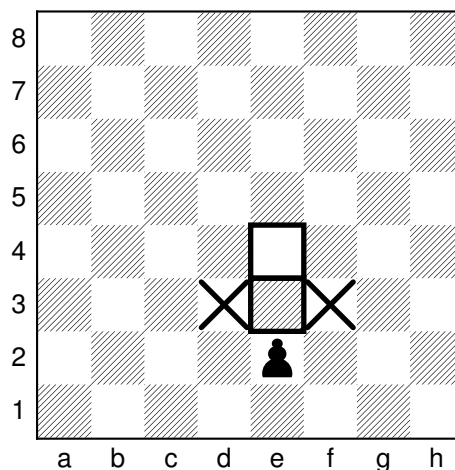


Fig. 1. Movimentação do Peão (Destacadas com um "x" as casas em que ele pode realizar captura de outras peças e com uma margem escura as casas em que ele pode se movimentar)

C. Cavalo

: Assim como qualquer outra peça, um cavalo tem como atributos seu ícone, sua coordenada, uma lista de possíveis movimentos, uma cor e um tabuleiro. Em um jogo de Xadrez, o cavalo se movimenta em forma de "L"^[3]. Ele nunca pode se movimentar de outras maneiras. Sua tomada é com base na lista de movimentos, não realizando nenhuma movimentação adicional. Desde que sua posição de destino esteja dentro do tabuleiro, seja no formato de "L" e não haja uma peça de mesma cor, o cavalo poderá se mover para tal posição. Para isso, os métodos presentes em **cavalo** são:

Atualizar lista de possíveis coordenadas: Como o cavalo pode andar para todos os lados, sem importar qual sua cor, não é necessário para sua classe métodos análogos à aqueles do peão para cada cor. Por isso, a partir da coordenada atual, esse método checa se é possível adicionar as coordenadas com base na movimentação em "L" do cavalo na lista de possíveis coordenadas. Para se mover de tal forma, o cavalo checa se poderá se mover de:

- Linha atual + 1 e Coluna atual - 2

- Linha atual + 2 e Coluna atual - 1
- Linha atual + 2 e Coluna atual + 1
- Linha atual + 1 e Coluna atual + 2
- Linha atual - 1 e Coluna atual + 2
- Linha atual - 2 e Coluna atual + 1
- Linha atual - 1 e Coluna atual - 2
- Linha atual - 2 e Coluna atual - 1

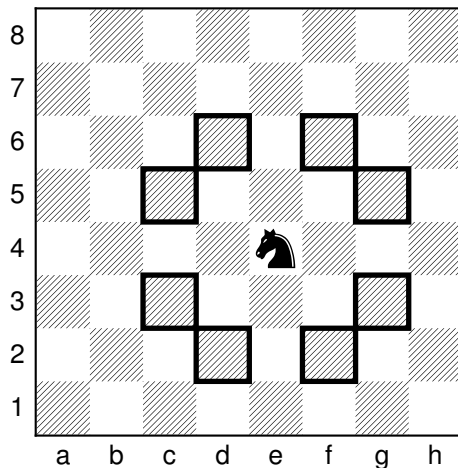


Fig. 2. Movimentação do Cavalo

D. Bispo

Assim como qualquer outra peça, um bispo tem como atributos seu ícone, sua coordenada, uma lista de possíveis movimentos, uma cor e um tabuleiro. Em um jogo de Xadrez, o bispo se movimenta nas diagonais^[3]. Ele nunca pode se movimentar de outras maneiras. Sua tomada é com base na lista de movimentos, não realizando nenhuma movimentação adicional. Desde que sua posição de destino esteja dentro do tabuleiro, esteja contido na sua diagonal e não haja uma peça de mesma cor, o bispo poderá se mover para tal posição. Para isso, os métodos presentes em **Bispo** são:

Atualizar lista de possíveis coordenadas: Esse método é dividido e modularizado em várias funções para que a movimentação do bispo seja realizada corretamente. Ela possui uma função **Dentro do tabuleiro**, que checa se uma linha e coluna estão dentro do tabuleiro como função helper. Além disso, também possui **Adicionar posição mais mais**, que adiciona todas as coordenadas possíveis na diagonal ++ utilizando recursividade. Além disso, também possui **Adicionar posição mais menos** que adiciona todas as coordenadas possíveis na diagonal +/- utilizando recursividade, somado a **Adicionar posição menos mais** que adiciona todas as coordenadas possíveis na diagonal -/+ utilizando recursividade além de **Adicionar posição menos menos**, que adiciona todas as coordenadas possíveis na diagonal -/- utilizando recursividade. Todas essas funções retornam somente quando a coordenada sai do tabuleiro, ou se a peça na posição desejada contém a mesma cor do bispo ou, caso todas checagens forem

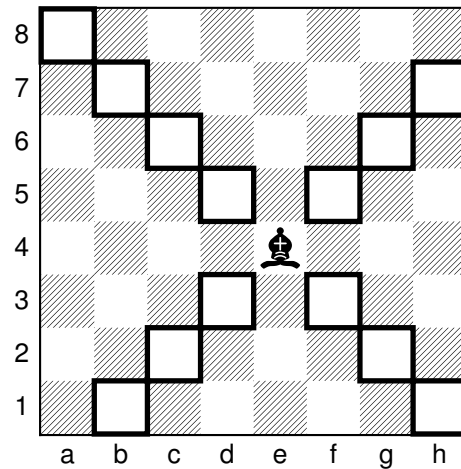


Fig. 3. Movimentação do Bispo

efetivadas, chama o método **Tentar adicionar e retorna**. Para cada um dos métodos, no fim deles, é chamado **Tentar adicionar** e a própria função, passando como parâmetros as linhas e colunas ++, +-, -+ ou --, analogamente ao nome.

E. Torre

Assim como qualquer outra peça, uma torre tem como atributos seu ícone, sua coordenada, uma lista de possíveis movimentos, uma cor e um tabuleiro. Além disso, a torre também deve ter um atributo já movimentou booleano visto que em uma partida de xadrez, o usuário pode realizar o **Roque**, jogada na qual, caso tanto o rei quanto a torre ainda não tiverem se movimentado, pode ser executada. Ela inverte a posição dessas peças. Em um jogo de Xadrez, a torre se movimenta em linhas retas, seja em uma linha ou coluna. Ela nunca pode se movimentar de outras maneiras^[3]. Sua tomada é com base na lista de movimentos, não realizando nenhuma movimentação adicional. Desde que sua posição de destino esteja dentro do tabuleiro, esteja contido na sua reta e não haja uma peça de mesma cor, a torre poderá se mover para tal posição. Para isso, os métodos presentes em **Torre** são:

Atualizar lista de possíveis coordenadas: Esse método é dividido e modularizado em várias funções para que a movimentação do cavalo seja realizada corretamente. Ela possui uma função **Dentro do tabuleiro**, que checa se uma linha e coluna estão dentro do tabuleiro como função helper. Além disso, o método possui funções auxiliares recursivas. Elas são: **Procurar linha esquerda:** Função que adiciona todas as coordenadas possíveis na esquerda da peça, utilizando recursividade. Também possui **Procurar linha direita**, que adiciona todas as coordenadas possíveis na direita da peça, utilizando recursividade, além de **Procurar coluna cima** que adiciona todas as coordenadas possíveis na acima da peça, utilizando recursividade e **Procurar coluna baixo**, que adiciona todas as coordenadas possíveis na abaixo da peça, utilizando recursividade. Todas essas funções retornam somente quando a

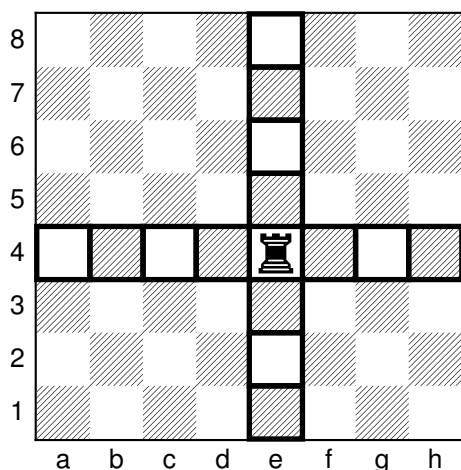


Fig. 4. Movimentação da Torre

coordenada sai do tabuleiro, ou se a peça na posição desejada contém a mesma cor da torre ou, caso todas checagens forem efetivadas, chama o método **Tentar adicionar** e retorna. Para cada um dos métodos, no fim deles, é chamado **Tentar adicionar** e a própria função, passando como parâmetros as linhas e colunas (linha atual e linha + 1, linha atual e linha - 1, linha - 1 e coluna atual, linha + 1 e coluna atual) - análogos ao nome.

F. Rainha

Assim como qualquer outra peça, uma rainha tem como atributos seu ícone, sua coordenada, uma lista de possíveis movimentos, uma cor e um tabuleiro. Em um jogo de Xadrez, a rainha une tanto os movimentos das diagonais quanto das retas. Ele nunca pode se movimentar de outras maneiras. Sua tomada é com base na lista de movimentos, não realizando nenhuma movimentação adicional^[3]. Desde que sua posição de destino esteja dentro do tabuleiro, esteja contido na sua diagonal ou reta e não haja uma peça de mesma cor, a rainha poderá se mover para tal posição. Para isso, os métodos presentes em **Rainha** são:

Atualizar lista de possíveis coordenadas: Junta as lógicas de **Torre e Bispo**: Esse método é dividido e modularizado em várias funções para que a movimentação do bispo seja realizada corretamente. Ela possui uma função **Dentro do tabuleiro**, que checa se uma linha e coluna estão dentro do tabuleiro como função helper. Além disso, também possui **Adicionar posição mais mais**, que adiciona todas as coordenadas possíveis na diagonal $++$ utilizando recursividade. Além disso, também possui **Adicionar posição mais menos** que adiciona todas as coordenadas possíveis na diagonal $+/-$ utilizando recursividade, somado a **Adicionar posição menos mais** que adiciona todas as coordenadas possíveis na diagonal $-/+$ utilizando recursividade além de **Adicionar posição menos menos**, que adiciona todas as coordenadas possíveis na diagonal $--$ utilizando recursividade. Todas essas funções

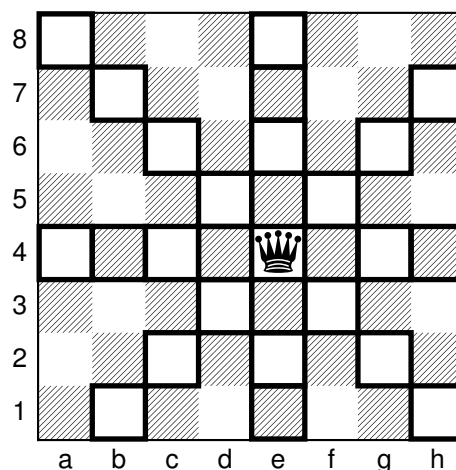


Fig. 5. Movimentação da Rainha

retornam somente quando a coordenada sai do tabuleiro, ou se a peça na posição desejada contém a mesma cor da rainha ou, caso todas checagens forem efetivadas, chama o método **Tentar adicionar e retorna**. Para cada um dos métodos, no fim deles, é chamado **Tentar adicionar** e a própria função, passando como parâmetros as linhas e colunas $++$, $+/-$, $-/+$ ou $--$, analogamente ao nome. Além disso o método possui funções auxiliares recursivas provindas da **Torre** Elas são: **Procurar linha esquerda**: Função que adiciona todas as coordenadas possíveis na esquerda da peça, utilizando recursividade. Também possui **Procurar linha direita**, que adiciona todas as coordenadas possíveis na direita da peça, utilizando recursividade, além de **Procurar coluna cima** que adiciona todas as coordenadas possíveis na acima da peça, utilizando recursividade e **Procurar coluna baixo**, que adiciona todas as coordenadas possíveis na abaixo da peça, utilizando recursividade. Todas essas funções retornam somente quando a coordenada sai do tabuleiro, ou se a peça na posição desejada contém a mesma cor da **Rainha** ou, caso todas checagens forem efetivadas, chama o método **Tentar adicionar** e retorna. Para cada um dos métodos, no fim deles, é chamado **Tentar adicionar** e a própria função, passando como parâmetros as linhas e colunas (linha atual e linha + 1, linha atual e linha - 1, linha - 1 e coluna atual, linha + 1 e coluna atual) - análogos ao nome.

G. Rei

Assim como qualquer outra peça, um rei tem como atributos seu ícone, sua coordenada, uma lista de possíveis movimentos, uma cor e um tabuleiro. Além disso, o rei também deve ter um atributo já movimentou booleano visto que em uma partida de xadrez, o usuário pode realizar o **Roque**, jogada na qual, caso tanto o rei quanto a torre ainda não tiverem se movimentado, pode ser executada. Ela inverte a posição dessas peças. Em um jogo de Xadrez, o rei se movimenta tanto nas diagonais quanto na retas, seja em uma linha ou coluna. porém somente em uma

casa. Ele nunca pode se movimentar de outras maneiras^[3]. Sua tomada é com base na lista de movimentos, não realizando nenhuma movimentação adicional. Desde que sua posição de destino esteja dentro do tabuleiro, esteja contido na sua reta e não haja uma peça de mesma cor, o rei poderá se mover para tal posição. O jogo acaba quando o rei está em estado de cheque mate, que acontece quando qualquer lugar que ele for, será capturado por peças inimigas. Essa checagem no entanto não é de responsabilidade da peça e será explicada mais à frente. Para isso, os métodos presentes em **Rei** são:

Atualizar lista de possíveis coordenadas: Percorre as casas ao seu redor do rei(linhas e diagonais) e chama o método **Tentar adicionar**.

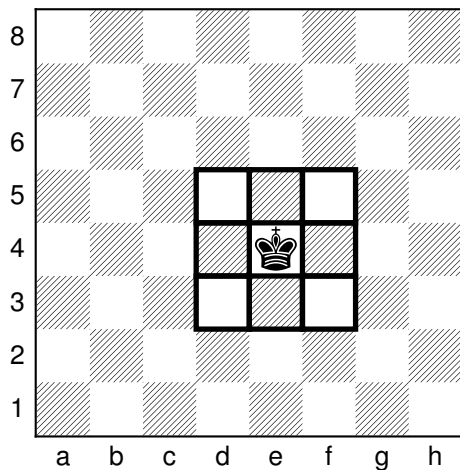


Fig. 6. Movimentação do Rei

H. Tabuleiro

Tabuleiro é a classe que contém e organiza as instâncias de peças, realiza movimentos e realiza as operações de roque e promoção do xadrez. Ela possui os getters para peças nas posições, para reis pretos e brancos e para peças de uma cor. Como atributos, possui uma matriz de casas, já que no xadrez alternamos entre quadrados pretos e quadrados brancos e uma matriz de peças, que vai sim, conter as peças do jogo. Quando uma peça é tomada ela deixa de existir no tabuleiro. Por isso, seus principais métodos são:

Gerar Matriz Tabuleiro: Que Retorna a disposição de casas do tabuleiro.

Gerar Configuração Inicial: Que Retorna a disposição inicial padrão das peças do tabuleiro, contendo, para cada lado, 8 peões, 2 bispos, 2 torres, 2 cavalos, 1 rei e 1 rainha.

Gerar String Tabuleiro: Gera uma string com o tabuleiro em sua posição atual. Percorre a matriz de peças e de casas, formatando os caracteres e acessando o ícone de cada uma das peças. Nas posições em que não há peças, coloca os elementos da matriz de casas.

Get Peça na Posição: Rece uma coordenada e retorna a peça que está em uma coordenada ou None, caso não haja nenhuma peça naquela posição.

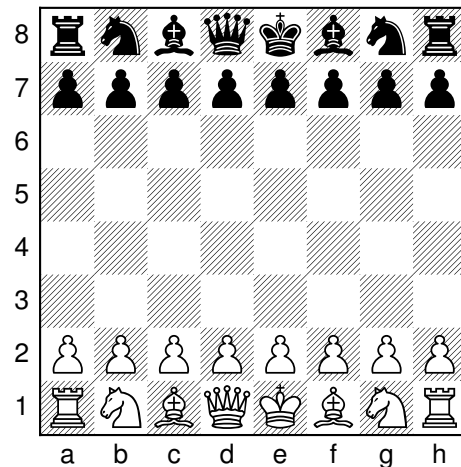


Fig. 7. Configuração padrão de um tabuleiro de xadrez^[3]

Mover peça: Recebe uma coordenada de origem, uma coordenada de destino e uma variável do tipo booleano chamada simulação, usada na lógica do cheque. Mas a função desse método é mover uma peça da coordenada de origem até a coordenada de destino. Após isso, chama o método **Atualizar todas as listas**, para garantir que todas as posições estejam atualizadas.

Voltar Movimento: Esse método recebe uma coordenada de origem e uma coordenada de destino, além de uma variável do tipo **Peça** ou **None**, chamada de Peça Tomada. Essa função retorna o jogo como ele estava antes de uma jogada. Então a peça tomada deixa de estar tomada e vai para onde era a posição de destino da peça que a tomou e a peça vai para sua posição inicial, antes de realizar a jogada. Caso a peça tomada seja **None**, a mesma lógica é aplicada mas nenhuma peça é colocada na coordenada de destino.

Atualizar todas as listas: Percorre toda a matriz de peças, limpando as listas de possíveis movimentos de cada peça e, para cada peça, atualiza a lista de possíveis coordenadas.

Promoção: Recebe um peão e uma nova peça que pode ser uma Rainha, Torre, Bispo ou Cavalo e define que a peça na posição prévia do peão será a nova peça.

Roque a esquerda: Checa a cor da peça e, se for branco, chama o movimentar peça passando como parâmetro o a coordenada do rei branco e a coordenada na qual deve ir, e a coordenada da torre e qual ela deve ir. O mesmo acontece para o Roque a direita mas com a lógica inversa.

Roque a Direita: Checa a cor da peça e, se for branco, chama o movimentar peça passando como parâmetro o a coordenada do rei branco e a coordenada na qual deve ir, e a coordenada da torre e qual ela deve ir. O mesmo acontece para o Roque a direita mas com a lógica inversa.

I. Coordenada

Classe para maior organização e facilitar as manipulações do usuário e do sistema em relação as peças. Possui como

atributos um inteiro para linha e um inteiro para coluna. Não possui maiores métodos.

J. Controladora

Por fim, A classe controladora é a principal classe do projeto. Ela é responsável pelo loop do jogo e checagens de jogadas, como cheque, mate e afogamento, promoção, roque. Também é responsável pela interação do Usuário com o programa e pela exibição dos dados. Ela tem como atributos um booleano para jogo em andamento, que será verdadeiro enquanto jogo estiver acontecendo, um lado que possui uma cor, inicialmente como branco, uma instância de tabuleiro, um booleano par cheque, uma peça dando cheque, que pode ser uma peça ou None e um ganhador que é uma cor. Por isso, possui como métodos:

Mostrar Tabuleiro: Imprime o tabuleiro

Coletar entradas: Coleta os dados da entrada padrão do jogador.

Alterar Lado: Altera o lado de quem está jogado. Se eram as pretas jogando, inverte o lado para as brancas jogarem e vice-versa.

Validar Jogada: Checa se uma jogada é válida ou não. Recebe uma coordenada de origem e uma coordenada de destino. O método checa se na coordenada de origem existe uma peça, checa também se na coordenada destino existe algum rei, já que reis não podem ser tomados, se a peça a ser movida é da cor de quem está jogando, se a coordenada de destino é diferente da coordenada de origem e também chama o método **Pode se mover para cá**, que se retornado verdadeiro evidencia que peça pode ir para tal posição. Inclusive é possível notar aqui um uso do polimorfismo, visto que esse mesmo método resulta em ações diferentes para **Peão** e para o restante das peças. Após isso, chama **Montar Jogada** e armazena seu retorno em uma variável chama peça tomada, chama **Rei em cheque** para checar se o rei está em cheque e voltamos movimento, chamando a função análoga. Temos que voltar o movimento porque estamos na realidade somente verificando se é possível realizar uma ação, mas ainda não a concretizamos.

Montar Jogada: O método monta uma jogada válida pra o jogo. Checa se roque para algum lado é possível. Pede a entrada para o usuário de qual tipo de jogada ele pretende fazer. Para a jogada normal, captura a entrada padrão do teclado do jogador que vem na forma de quatro inteiros: uma linha de origem + coluna de origem e uma linha de destino + coluna de destino. Após isso, o método chama **Validar Jogada**. Caso a instrução seja de roque para a esquerda, ela chamará **Roque a esquerda** e caso a instrução seja roque a direita, será chamado **Roque a direita**.

Roque possível: Checa se há possibilidade de fazer o roque, retornando dois booleanos, um indicando se é possível fazer o roque para a esquerda e outra para a direita, checando no meio tempo se algum dois reis vai entrar em cheque, ou se todas as coordenadas passadas não possuem peças em suas posições e checando se o roque é possível para uma lista de coordenadas

entre o rei e a torre e sabendo se a torre já se movimentou ou não.

Iniciar: É o loop principal do jogo. Enquanto o jogo está em andamento, ela descobre o lado do adversário, chama função de mostrar o tabuleiro, chama **Montar Jogada**, após montá-la, a executa, chamando **Mover peça**. É importante ressaltar que **Mover peça** somente move peças de um lugar para o outro, enquanto toda a parte de checagem é realizada por **Validar Jogada**, que é chamada por **Montar Jogada**. Depois de realizar a jogada, confere se a peça movida é um peão e se ela está elegível para promoção. Para isso, chama **Promover se possível**. Depois, chama **Averiguar Cheque**, e se o lado adversário estiver sem movimentos legais, chamando **Sem movimentos legais**, será dado o cheque mate. Caso o adversário não esteja em cheque, mas sem movimentos, o jogo é encerrado por afogamento. Após todas essas chamadas, o método chama **Trocar lado** e a vez é dada para o oponente.

Promover se Possível: Checa se é possível promover um peão e a promove case seja. Confere se o peão está na linha lado oposto ao que começou.

Promoção: Promove um peão para uma rainha, torre, bispo ou cavalo. Troca o peão naquela posição por uma nova peça escolhida pelo Usuário.

Checar Cheque: Checa se algum dos lados está em cheque, retornando a peça que está deixando o rei em cheque além da informação se o cheque ocorre ou não.

Rei em Cheque: Verifica se o rei de um lado está em cheque.

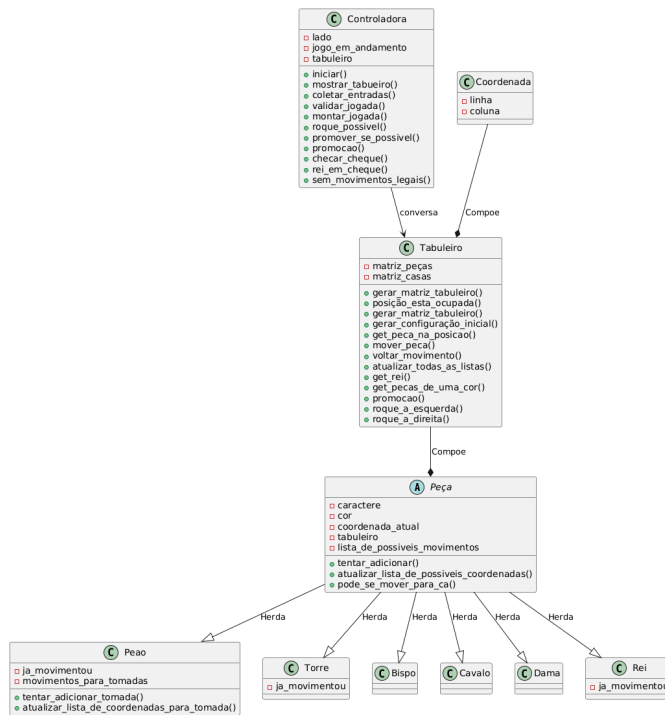
Sem movimentos legais: Recebe uma cor e verifica se, para ela existem movimentos legais para as peças de um lado. Percorre todo o tabuleiro e, para cada peça, realiza a checagem se ela possui algum movimento em sua lista de possíveis movimentos que é legal. Para o peão, realiza a verificação adicional das movimentações para tomada.

IV. CONCLUSÃO

Em conclusão, esse trabalho foi um meio de implementar o jogo de xadrez no terminal, utilizando-se da linguagem **Python** e dos conceitos de análise, projeto e POO vistos na disciplina. Utilizamos também os slides da disciplina de Processos e Métodos em Engenharia de Sistema para o planejamento e organização de cada etapa de realização do trabalho. Nesse projeto, os maiores desafios foram com relação as especificidades do jogo de xadrez, especialmente envolvendo o peão que é a peça que possui mais distinções das demais, sendo seu método de "comer" outras peças o único indireto e pelo fato de ser capaz de ser promovido a qualquer outra peça caso chegue ao fim do lado oposto do tabuleiro. Outro ponto importante foram as alterações ao longo do desenvolvimento do projeto, inicialmente gostaríamos de simular um jogo xadrez de forma mais dinâmica semelhante aos campeonatos de xadrez moderno, como tempo de jogadas para cada jogador, entretanto isso gerou varias complicações na implementação especialmente nos casos de vitória por tempo de um dos jogadores.

V. APÊNCIDE

Segue aqui o diagrama UML que auxiliou o grupo durante o desenvolvimento do projeto:



REFERENCES

- [1] Real, Luiza, Slides da Disciplina de Análise, Projeto e Programação Orientada a Objetos. 2026.
- [2] Batista, André, Liddy, Ana, Slides da Disciplina de Processos e Métodos em Engenharia de Sistemas. 2026.
- [3] FACCHINI, M. Xadrez: aprenda regras e movimentos (com imagens). Disponível em: <https://www.todamateria.com.br/xadrez/>.